資訊工程導論
第0章 :緒論(Introduction)

本講義未經同意請勿自行翻印

教科書: Computer Science an overview, 13th Edition, J. Glenn Brookshear and D. Brylow, Pearson, Addison Wesley, Pub. (滄海圖書代理) 中英文版

第 0 章 緒論

- 0.1 演算法的作用
- 0.2 計算機的過往歷史
- 0.3 包羅萬象的計算機科學主題

© 滄海圖書

課程目標(Lesson Objectives)

完成本 Module 後，學生能夠：

- 定義 algorithm、program、programming、software 與 hardware，並說明彼此關係。
- 以實例說明 algorithm 如何解決日常及 computing problems。
- 依時間順序描述並比較 computing technology 的重要發展階段。
- 辨識並連結本課程的主要 computer science themes。
- 分析 一項 computing technology 可能造成的 social, ethical, privacy 或 sustainability impacts。

© 滄海圖書



0.1 演算法的作用
Terminology(專門用語)

- **Algorithm（演算法）**

  A sequence of well-defined steps for solving a problem.

- **Program（程式）**

  Instructions written in a programming language to implement one or more algorithms.

- **Programming（程式設計）**

  The process of designing, writing, testing and debugging programs.

- **Software（軟體）**

  Programs and related data, configuration and documentation that enable a computer system to perform tasks.

- **Hardware（硬體）**

  The physical components of a computer system.



© 滄海圖書

Examples

- **Everyday Algorithm**
  - Making instant noodles
- **Simple Computational Algorithm**
  - Find the larger of two numbers
- **Algorithm representation(表示)**
  - Natural language(自然語言) → pseudocode (虛擬碼)→ flowchart
- **GCD**
  - Euclidean Algorithm (歐幾里得算法)

-> Python implementation(實作)

© 滄海圖書



演算法的作用

- **定義**：解決問題的一系列步驟。
- **傳統案例**：輾轉相除法 (GCD)、長除法。
- **現代案例**：
  - Google 搜尋排序
  - YouTube/Netflix 推薦系統
  - ChatGPT 的語言生成邏輯
  - AI 下棋 (AlphaGo)

五大特性:

輸入(Input)
輸出(Output)
確定性(Definiteness)
有效性(Effectiveness)
有限性(Finiteness)

- **互動問題**：你覺得手機上的「推薦內容」是如何被演算法決定的？

© 滄海圖書

圖0.2 用來找兩個正整數的最大公因數的輾轉相除法

描述：這演算法是輸入兩個正整數，並計算出兩數的最大公因數。

程序：

步驟 1. 分別假設 M 為兩數中的較大數，N 為較小數。

步驟 2. 將 M 除以 N，餘數為 R。

步驟 3. 若 R 不為 0，將 M 改設為 N 的值，N 為 R 的值，然後回到步驟 2。若 R 為 0，最大公因數即為 N 的值。

Terms:
Greatest common divisor(GCD):最大公因數
Euclidean Algorithm(歐幾里得算法)/Long division(長除法)
Pseudocode(虛擬碼)

© 滄海圖書

GCD Example

1. M=24 N=16
2. 24%16=8->R
3. if R not = 0 then
   M = N =16
   N=R=8
   goto 2
   else
   if R = 0 then GCD =N

```
[10]: m= 24; n=16
```

```
[11]: r = m%n
      while r !=0:
          m=n
          n=r
          r= m%n

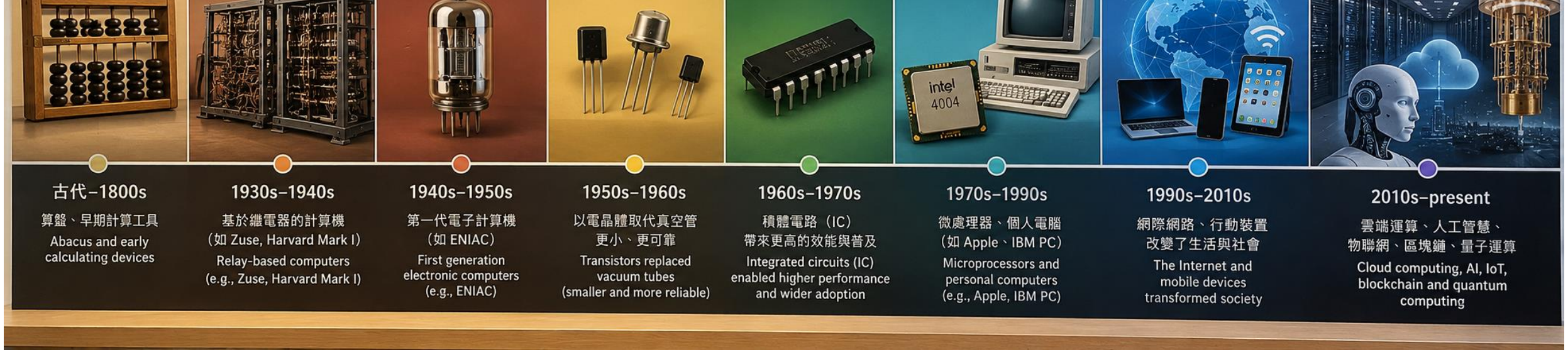
      print( n )
```

```
8
```

Step	M	N	R = M mod N
1	24	16	8
2	16	8	0

© 滄海圖書

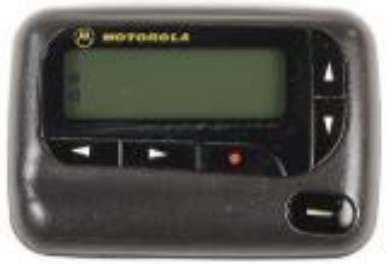




- **古典時期**：算盤、齒輪機械
  - (Pascal(1623–1662)、Leibniz(1646–1716)、Babbage(1791–1871))
    - Jacquard (1752–1834, loom)
    - Hollerith(1860-1929, Census)
    - Ada(1970)
- **早期電腦**：
  - 電子機械式
    - Stibitz(1940 in Bell Lab)->電子機械計算機
    - Mark I(1944, Aiken and IBM 工程師)
  - 真空管
    - Atanasoff and Berry(1937~1941)->第一部真空管計算機
    - Colossus(Flowers, 1943–1944) ->二戰解德軍密文
    - ENIAC(Mauchly and Eckert, 1945)
  - 積體電路(Kilby, ~1970)
    - Apple (1976(
    - IBM PC (1981)
- **千禧年後**：
  - 智慧型手機、嵌入式系統

- **現代趨勢 (2000+ 年代)**：
  - 雲端運算 (Cloud Computing)
    - AWS(Amazon Web Services)、Google Cloud、Microsoft Azure
  - 人工智慧與深度學習 (AI & Deep Learning)
    - AlphaGo、ChatGPT、MidJourney
  - 物聯網 (IoT) 與 5G/6G
    - 自駕車、AR/VR、遠距醫療
  - 區塊鏈與加密貨幣
    - 比特幣 (Bitcoin)、以太坊 (Ethereum)、非同質化代幣(Non-Fungible Token; NFT)。
  - 量子運算 (Quantum Computing)
    - Google Sycamore、IBM Quantum

Examples















8吋、5.25吋及3.5吋磁片

研發日期	1969年（8吋）
	1976年（5¼吋）
	1984年（3½吋）

~wikipedia

© 滄海圖書

問題討論

- 你覺得電腦史上哪個階段對人類影響最大？為什麼？
- 如果沒有智慧型手機，你的生活會有什麼不同？
- 量子電腦普及後，你認為哪個行業最容易被改變？
- 摩爾定律(Moore’s Law)?

© 滄海圖書

總結

- 電腦史主要可分為四大階段：古典 → 早期電子 → 個人化 → 行動化。
- 現代計算機發展趨勢：**雲端、AI、IoT、區塊鏈、量子**。
- 電腦的角色從「計算工具」進化為「社會運作核心基礎」。

© 滄海圖書

休息片刻，來點輕鬆! 影片欣賞

- **History of Computers | From 1930 to Present**

https://www.youtube.com/watch?v=gjVX47dLlN8

- Raspberry Example
  - https://en.wikipedia.org/wiki/Raspberry_Pi

© 滄海圖書



想像力 V.S.創造力

- 脫離身體也能控制爬行！中國智能仿生手讓“意念控制”照進現實

https://www.facebook.com/reel/1817368149440100

- **What If Robots Starts Living In Your House**

https://www.facebook.com/fossbytes/videos/858607413155983/?rdid=MAlX7G64ypNKCWg5#

© 滄海圖書

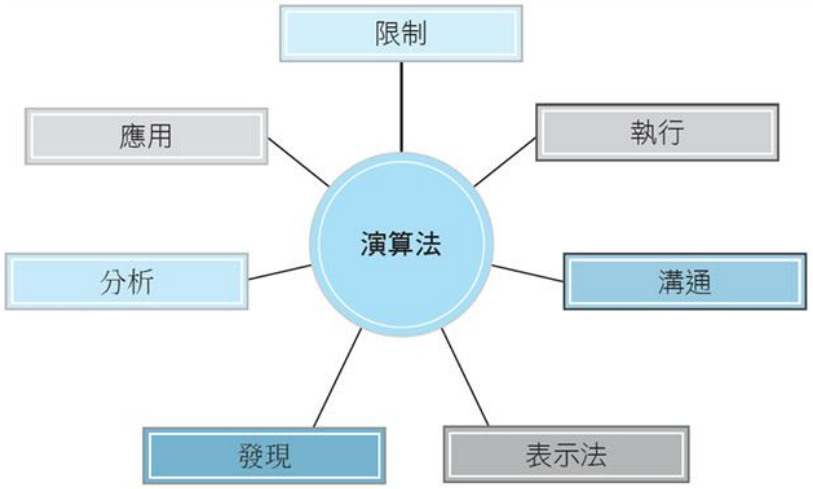
0.4 包羅萬象的(overarching)計算機科學主題

- 計算機技術是作為現代社會一份子的基礎
- 本書涵蓋計算機科學應用和相關後果
- 計算機科學的七大想法(big ideas)：傳統七大主題
  - Algorithms(演算法), Abstraction(抽象化), Creativity(創造性), Data(資料), Programing(程式化), Internet(網際網路)and Impact(影響力)
- 目前, 另括展至九大主題
  - **人工智慧與機器學習 (AI & Machine Learning)**
  - **數位永續 (Digital Sustainability & Green IT)**

© 滄海圖書

演算法(Algorithm)

1. 哪些是能夠被演算法解決的問題？
2. 如何使演算法的發現變得更容易？
3. 如何能夠改進演算法在程式語言中的表示和訊息傳遞技術？
4. 如何分析和比較不同的演算法之間的特性？
5. 如何利用演算法來模擬人類的智慧行為？
6. 如何使用演算法來操控資訊？
7. 演算法的相關應用如何影響這個社會？



© 滄海圖書

抽象化(Abstraction)

抽象化

- 實體的外在性質與內部組成的區別

抽象化工具

- 一種可直接使用的元件，而不需顧慮其內部構造

© 滄海圖書

創造性(Creativity)

- 計算機科學本質上是非常具有創造性
  - 發明並應用演算法是人類的行為之一
  - 拓展既有的表達形式
  - 展開數位表達形式的新模式
- 建立大型軟體系統像是構想一個全新的雕刻品

© 滄海圖書

資料(Data)

- 電腦能夠表達離散和數位化的任何資訊
- 演算法可處理並轉換資料
  - 搜尋字串或特徵值
  - 建構模擬
  - 產生知識和看法
- 資料驅使現代科學的進展

© 滄海圖書

關於資料的一些問題

- 電腦是如何儲存數位資料？
  - 數值、文字、圖像、聲音和影像
- 電腦是如何將真實世界中的類比資料估算出數位資料？
- 電腦是如何偵測並預防資料中的錯誤？
- 在我們手邊不斷成長和交錯的數位資料其結果(ramifications)會是什麼？

© 滄海圖書

程式化(Programming)

- **基本運作流程**：Fetch → Decode → Execute → Write-back。
- **程式語言演進**：C → Java → Python → JavaScript
- Programming Approaches and Tools:
  - Traditional Coding-> 低程式碼平台 (Low-code)->AI-assisted Coding -> Vibe Coding -> Agentic Coding。
    - Examples: Codex and other AI coding agents
- **Python 的角色**：AI、資料科學的核心工具。

人類的想法轉化成可執行的電腦演算法

- 傳統: Human Idea → Algorithm → Program → Execution → Output
- AI 時代: Human Intent → AI Coding Agent → Code → Test → Human Review → Software

© 滄海圖書

Interesting Questions?

- If AI can write the code, do we still need to learn programming?
  - 要能判斷:
    - 這個 algorithm 對不對?

      程式有沒有 bug?

      產生的結果可信嗎?

      有沒有 security risk?

      有沒有 privacy problem?

© 滄海圖書



程式化的一些問題

- 程式是如何建立的？
- 程式會發生什麼樣的錯誤？
- 程式中的錯誤如何發現和修補？
- 現代程式的錯誤會有什麼影響？
- 程式要如何記錄和評估？

© 滄海圖書

網際網路(Internet)

- **核心功能**：資訊儲存(store)、擷取(retrieve)、分享(share information)。
- **安全與隱私**：加密技術、網路安全(cybersecurity)。
- **現代趨勢**：
  - 雲端服務 (AWS, GCP, Azure)
  - 社群媒體與演算法
  - 資訊戰、假新聞與 AI 假影像 (Deepfake)

AWS (Amazon Web Services)
Google Cloud Platform (GCP)

- **討論題**：如果你在社群媒體看到一段非常令人震驚的影片，你要如何判斷它是真實的，還是 AI 產生的 deepfake?

© 滄海圖書

影響力(Impact)與社會議題

- **傳統議題**：隱私(privacy)、品格道德(Character-based/Virtue-based ethics)、責任(duty-based)、後果(consequence-based) 、契約倫理(contract-based)。
- **當代新挑戰**：
  - AI 偏見與公平性 (Bias & Fairness)
  - 就業市場的影響 (自動化與 AI 替代)
  - AI Deepfake 與資訊可信度
  - AI for Good → 可持續發展 (AI + SDGs)

Sustainable Development Goals(SDGs)

討論題：如果 AI 判斷一名求職者是否合格，可能會出現什麼問題？

© 滄海圖書

其他主題

- 人工智慧與機器學習 (AI & Machine Learning(ML))
- 數位永續與綠色資訊科技(Digital Sustainability & Green IT)
- 串起九大主題：一條實務學習路徑
  - **從需求出發**（創造性）→ **抽象介面**（抽象化）→ **設計演算法/資料結構**（演算法/資料）
  - **把解法寫成可維護的程式**（程式化）→ **部署到網路/雲端**（網際網路）
  - 若含智慧功能則引入 **AI/ML** → 全程評估 **影響力(impact)/倫理(Ethics)** → 檢視 **永續與能效**

© 滄海圖書



Example: Google Maps

Big Idea	Google Maps Example
Algorithm	Find the best route
Abstraction	User sees the route, not internal calculations
Creativity	Design new navigation functions
Data	Maps, GPS, traffic data
Programming	Implement functions
Internet	Retrieve real-time information
Impact	Privacy, surveillance, accessibility
AI	Traffic prediction
Sustainability	Energy and transport efficiency

© 滄海圖書



Ethics(倫理) Questions

1. Algorithm： Could a recommendation algorithm be unfair?
2. Data： Who owns your data?
3. Internet： Who can see your information?
4. AI： Can an AI decision be trusted?

© 滄海圖書



Conclusion

- 演算法定義為一件工作如何被執行的一連串[redacted]。
- 電腦的演進主要經過以下6個階段:電腦的演進主要經過以下6個階段:[redacted]
- 包羅萬象的計算機科學主題，包含以下7項:[redacted]
- 演算法涵蓋的7項範疇為:[redacted]
- 哲學家提出電腦科學帶給社會4種道德倫理影響的標準為:[redacted]為主的道德倫理。

© 滄海圖書

影片欣賞

- **Computer science ethics education: The case for ethics integration in the tech classroom**

https://www.youtube.com/watch?v=LYRKqnLeIDo

© 滄海圖書

最後

- If AI Can Code, Why Learn Computer Science?
  - AI can generate code, but
  - Computer scientists decide what should be built and whether it works. So,
  - The future skill is not just coding — it is problem solving, system thinking, and AI verification.

Don’t compete with AI at typing code.
Learn to think at a level above the code.

© 滄海圖書

